

- 手写快读，快写
- $(a/b)\%mod$
- 离散化
- head
- 重写结构体运算符
- sosdp
- pbds库

手写快读，快写

```
int read() {
    int res = 0;
    char c = getchar();
    while(!isdigit(c)) c = getchar();
    while(isdigit(c)) res = (res << 1) + (res << 3) + c - 48, c = getchar();
    return res;
}

void printi(int x) {
    if(x / 10) printi(x / 10);
    putchar(x % 10 + '0');
}
```

$(a/b)\%mod$

$$(a/b)\%mod = (a * ksm(b, mod-2))\%mod$$

离散化

```
std::vector<int> tmp(arr); // tmp 是 arr 的一个副本
std::sort(tmp.begin(), tmp.end());
tmp.erase(std::unique(tmp.begin(), tmp.end()), tmp.end());
for (int i = 0; i < n; ++i)
    arr[i] = std::lower_bound(tmp.begin(), tmp.end(), arr[i]) - tmp.begin();
```

head

```
#include <bits/stdc++.h>
#include <ext/pb_ds/priority_queue.hpp>
using namespace std;
using namespace __gnu_pbds;
#define int long long
#define ld long double
#define endl "\n"
#define fr(i, n) for (int i = 1; i <= n; i++)
#define vec vector<long long>
#define lowbit(x) ((x) & (-x))
#define vecp vector<pair<int, int>>
#define all(a) a.begin(), a.end()
const int N = 1e3 + 5;
const int MOD = 998244353;
const int INF = 1e18;
int n;
int gcd(int a, int b) { return b ? gcd(b, a % b) : a; }
long long ksm(long long a, long long b) {
    long long res = 1;
    while (b) {
        if (b & 1)
            res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}
void Add(int &x, int y) { ((x += y) >= MOD) ? (x -= MOD) : 0; }
int Add(int x, int y) { return (x += y >= MOD) ? x - MOD : x; }
void Mul(int &x, int y) { x = (x * y + MOD) % MOD; }
int Mul(int x, int y) { return (x * y + MOD) % MOD; }
void init() {}
void solve() {}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

重写结构体运算符

```

struct node{
    int a,b;
    bool operator < (const node& x) const{
        return a>x.a;
    }
};

```

sosdp

```

for(int k=0;k<=19;k++)
    for(int i=0;i<=maxn;i++)
        if((i>>k)&1) g[i] += g[i^(1<<k)];

```

pbds库

```

#include<bits/stdc++.h>
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;
// 由于面向OIer，本文以常用堆：pairing_heap_tag作为范例
// 为了更好的阅读体验，定义宏如下：
using pair_heap = __gnu_pbds::priority_queue<int>;
pair_heap q1; // 大根堆，配对堆
pair_heap q2;
pair_heap::point_iterator id; // 一个迭代器

int main() {
    id = q1.push(1);
    // 堆中元素：[1];
    for (int i = 2; i <= 5; i++) q1.push(i);
    // 堆中元素：[1, 2, 3, 4, 5];
    std::cout << q1.top() << std::endl;
    // 输出结果：5;
    q1.pop();
    // 堆中元素：[1, 2, 3, 4];
    id = q1.push(10);
    // 堆中元素：[1, 2, 3, 4, 10];
    q1.modify(id, 1);
    // 堆中元素：[1, 1, 2, 3, 4];
    std::cout << q1.top() << std::endl;
    // 输出结果：4;
    q1.pop();
    // 堆中元素：[1, 1, 2, 3];
    id = q1.push(7);
    // 堆中元素：[1, 1, 2, 3, 7];
    q1.erase(id);
}

```

```
// 堆中元素 : [1, 1, 2, 3];
q2.push(1), q2.push(3), q2.push(5);
// q1中元素 : [1, 1, 2, 3], q2中元素 : [1, 3, 5];
q2.join(q1);
// q1中无元素, q2中元素 : [1, 1, 1, 2, 3, 3, 5];
}
```

pbds库使用时记得include, 它不在bits/stdc++中,同时可以同时使用using namespace std和using namespace __gnu_pbds

堆的自定义排序需要包装一个struct, 如下

```
struct Cmp {
    bool operator()(const int &a, const int &b) const {
        if (dep[a] != dep[b]) return dep[a] < dep[b];
        return a < b;
    }
};

using Heap = __gnu_pbds::priority_queue<int, Cmp, pairing_heap_tag>;

vector<Heap> root(n + 1);
```